

Movie Ticket Reservation System: Requirements Engineering Document

Maria Vale 1251223

Afonso Maia 1242115

ISEP – Instituto Superior de Engenharia do Porto
Fundamentos de Desenvolvimento de Software (FSOFT) – 2025/26

Abstract. This document presents the requirements engineering artifacts for a Movie Ticket Reservation System developed in C++ as part of the FSOFT course at ISEP. The system allows users to register an account, log in, check movie schedules and prices, and make reservations by choosing a movie, specifying the number of tickets, selecting chairs, and entering payment details. The document covers Requirements Engineering concepts, User Stories, Domain Model, Functional and Non-Functional Requirements (FURPS+), Use-Case Model with specifications, System Sequence Diagrams, and a Glossary.

Keywords: Requirements Engineering, User Stories, Use Cases, Domain Model, FURPS+, SSD, Glossary, Movie Ticket Reservation, C++

1 Introduction

This document defines and documents the requirements for a Movie Ticket Reservation System to be implemented in C++. The system provides the following main functionalities to its users:

Register and Login: Users create an account and authenticate before accessing system features.

Check Schedule: Users can view the schedule of available movies, including dates, times, and screening rooms.

Check Prices: Users can consult ticket prices for different movies and seat types.

Reservation: Users follow a guided flow: choose a movie, specify the number of tickets, select chairs from the seating map, enter payment details, and receive a reservation confirmation screen.

Exit: Users can log out and exit the system.

The requirements engineering process follows the methodology taught in FSOF, producing the following artifacts: Requirements Engineering overview, User Stories, Domain Model, Functional and Non-Functional Requirements (FURPS+), Use-Case Model, System Sequence Diagrams, and a Glossary.

2 User Stories

User stories are business needs, not requirements in the traditional sense. They are oriented toward the user and a business need. A user story describes a business need, not the system's functionality — the functionality that fulfills the need is the Development Team's job. A user story is an informal, general explanation of a software feature written from the perspective of the end user, following the format: As a <role>, I want to <action>, so that <benefit>.

Each story includes acceptance criteria — detailed conditions that a feature must meet to be considered complete. Acceptance criteria are more technical, offering a checklist that ensures the feature behaves as intended. They should be testable and reveal straightforward yes/no or pass/fail results.

2.1 US01 – Register

User Story	As a new user, I want to register an account so that I can access the reservation system.
Acceptance Criteria	1. The system requests a username, email, and password. 2. The password must have a minimum of 8 characters, at least 1 uppercase, 1 lowercase, and 1 digit. 3. The system validates that the username and email are not already in use. 4. Upon successful registration, the system confirms the account creation.

Table 1 - US01 - Register

2.2 US02 – Login

User Story	As a registered user, I want to log in so that I can access the system features.
Acceptance Criteria	1. The system requests username and password. 2. The system validates the credentials against the stored data. 3. Upon successful login, the system displays the main menu. 4. After 3 failed attempts, the account is temporarily locked.

Table 2 - US02 - Login

2.3 US03 – Check Schedule

User Story	As a user, I want to check the movie schedule so that I can see what movies are available and when.
Acceptance Criteria	1. The system displays a list of movies with their showtimes.
	2. Each entry shows the movie title, date, time, and screening room. 3. Only future showtimes are displayed.

Table 3 - US03 - Check Schedule

2.4 US04 – Check Prices

User Story	As a user, I want to check ticket prices so that I can decide which movie and seat type fits my budget.
Acceptance Criteria	1. The system displays the price for each movie. 2. Prices are shown per seat type (e.g., regular, VIP). 3. Any active promotions or discounts are displayed.

Table 4 - US04 - Check Prices

2.5 US05 – Make a Reservation

User Story	As a user, I want to make a reservation so that I can guarantee my seats for a movie.
Acceptance Criteria	1. The user selects a movie from the available list. 2. The user specifies the number of tickets. 3. The system displays the seating map for the chosen screening. 4. The user selects the desired chairs from available seats. 5. The system requests payment details (card number, expiry, CVV). 6. Upon successful payment, a reservation confirmation screen is displayed with a unique booking reference. 7. The reserved chairs are marked as unavailable for other users.

Table 5 - US05 - Make a Reservation

2.6 US06 – Exit

User Story	As a user, I want to exit the system so that I can safely end my session.
Acceptance Criteria	1. The system logs the user out. 2. Any unsaved reservation in progress is discarded. 3. The system returns to the login/welcome screen.

Table 6 - US06 - Exit

3 Requirements Engineering

3.1 What is a Requirement?

A requirement is a necessary attribute in a system, a statement that identifies a capability, characteristic, or quality factor of a system in order for it to have value and utility to a customer or user. Requirements are important because they provide the basis for all of the development work that follows. Once the requirements are set, developers initiate the other technical work: system design, development, testing, implementation, and operation.

A requirement is a condition or capability required by a stakeholder to solve a problem or achieve an objective, while a need is a high-level representation of the requirement. The need is the end result or purpose — it is the why we are doing this.

3.2 Requirement vs Need

The following table illustrates the distinction between a need and the requirements that fulfill it, applied to our Movie Ticket Reservation System:

Need (I need to...)	Requirements
Reserve seats for a movie online	The system shall display a list of available movies.
	The system shall show available showtimes for a selected movie.
	The system shall display a seating map with available and occupied chairs.
	The system shall allow the user to select specific chairs.
	The system shall request payment details to confirm the reservation.
	The system shall generate a unique booking reference.
	The system shall display a reservation confirmation screen.

Table 7 - Table of Requirements

4 Domain Model

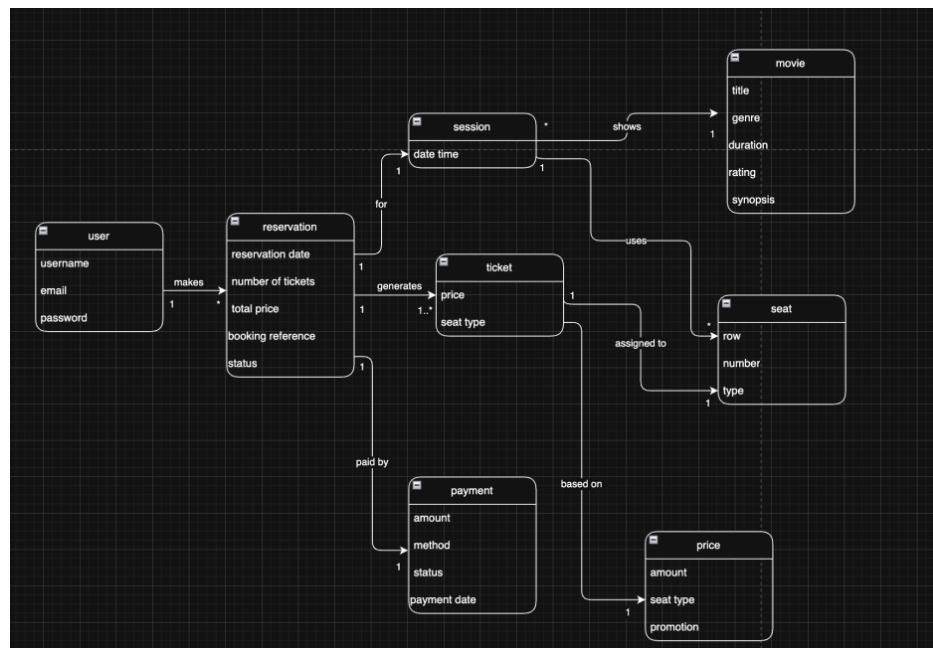


Figure 1 - Domain Model

5 Requirements Specification

Requirements are classified into functional and non-functional categories. Functional requirements describe what the system must do. Non-functional requirements are categorized using the FURPS+ model.

5.1 Functional Requirements

ID	Description
FR01	The system shall allow a new user to register with username, email, and password.
FR02	The system shall validate that username and email are unique during registration.
FR03	The system shall enforce password rules (min. 8 chars, 1 uppercase, 1 lowercase, 1 digit).
FR04	The system shall allow a registered user to log in with username and password.
FR05	The system shall lock the account after 3 consecutive failed login attempts.
FR06	The system shall display the movie schedule with title, date, time, and room.
FR07	The system shall allow filtering the schedule by date or movie title.
FR08	The system shall display ticket prices per movie and seat type.
FR09	The system shall allow the user to select a movie for reservation.
FR10	The system shall allow the user to specify the number of tickets.
FR11	The system shall display the seating map showing available and occupied chairs.
FR12	The system shall allow the user to select specific chairs from available seats.
FR13	The system shall request payment details (card number, expiry date, CVV).
FR14	The system shall validate payment details before processing.
FR15	The system shall generate a unique booking reference upon successful reservation.
FR16	The system shall display a reservation confirmation screen with booking details.
FR17	The system shall mark reserved chairs as unavailable for other users.
FR18	The system shall allow the user to exit and log out safely.

Table 8 - Functional Requirements

5.2 Non-Functional Requirements (FURPS+)

ID	Category	Description
NFR01	Functionality	The system shall require login before accessing schedule, prices, or reservation features.
NFR02	Functionality	The system shall store user passwords in an encrypted format.
NFR03	Functionality	The system shall log all reservation transactions for auditing.
NFR04	Usability	The seating map shall visually distinguish available, occupied, and selected chairs.
NFR05	Usability	The system shall provide clear error messages for invalid inputs.
NFR06	Usability	The main menu shall clearly present all options: Check Schedule, Check Prices, Reservation, Exit.
NFR07	Reliability	The system shall not lose confirmed reservation data in case of unexpected termination.
NFR08	Reliability	The system shall handle concurrent seat selections to prevent double-booking.
NFR09	Performance	The system shall display the schedule and prices without delay.

NFR10	Performance	The seating map shall update in real time after a chair is selected.
NFR11	Supportability	The system shall be developed in C++ with modular architecture.
NFR12	Supportability	The system shall include unit tests for registration, login, and reservation logic.

Table 9 - Non-Functional Requirements

Additional Constraints (+). The + in FURPS+ covers design, implementation, interface, and physical constraints.

Constraint Type	Description
Design	The system shall follow a layered architecture (UI, business logic, data persistence).
Implementation	The system shall be implemented in C++ using standard libraries.
Interface	The system shall use a console-based (text) user interface.
Physical	The system shall run on standard desktop hardware (Windows/Linux).

Table 10 - Additional Constraints

6 Use-Case Model

Use case diagrams are used to gather usage requirements of a system and to identify functions and how roles interact with them. The use-case model identifies the actors interacting with the system and the use cases representing system functions.

6.1 Actors

An actor in a use case diagram is any entity that performs a role in one given system. This could be a person, organization, or an external system.

Actor	Description
Unregistered User	A person who has not yet created an account. Can only register.
User	A registered and logged-in person who can check schedules, prices, make reservations, and exit.

Table 11 - Actors

6.2 Use Cases

A use case represents a function or an action within the system. The system (or subject) is used to define the scope of the use case. An association is the relationship between an actor and a use case, indicating that an actor can perform a certain action.

ID	Use Case	Actor(s)
UC01	Register	Unregistered User
UC02	Login	Unregistered User
UC03	Check Schedule	User
UC04	Check Prices	User
UC05	Make a Reservation	User
UC06	Exit	User

Table 12 - Use Cases

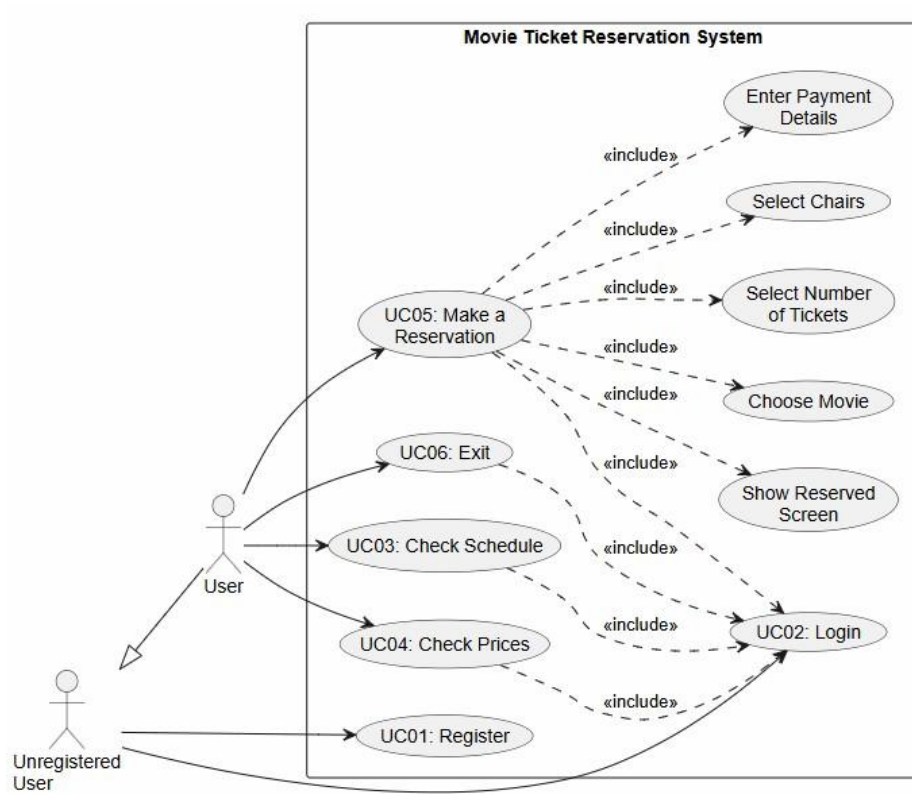


Figure 2 - Use Case Model

6.3 Use Case Specification UC01: Register

Actor	Unregistered User
Use Case Name	Register
Description	Allows a new user to create an account to access the system.
Precondition	The user does not have an existing account.
Postcondition	A new user account is created and stored in the system.
Main Flow	1. The system displays the registration form. 2. The user enters a username, email, and password. 3. The system validates that the username and email are unique. 4. The system validates the password meets the rules (min. 8 chars, 1 upper, 1 lower, 1 digit). 5. The system creates the account. 6. The system displays a success message and redirects to the login screen.
Alternative Path	3a. The username or email is already in use. → The system displays an error. Resume at step 2. 4a. The password does not meet the requirements. → The system displays an error. Resume at step 2.
Exceptions	At any point, the user may cancel the registration. → The system returns to the welcome screen.

Table 13 - UC01: Register

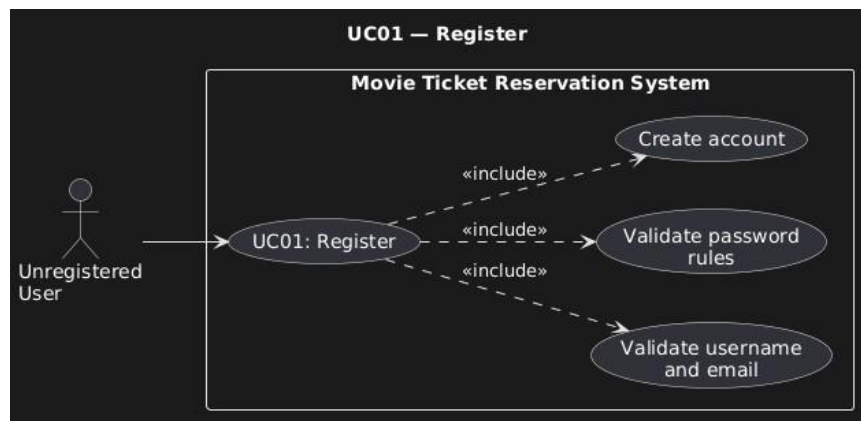


Figure 3 - UC1 Register

—

6.4 Use Case Specification UC02: Login

Actor	Unregistered User (with existing account)
Use Case Name	Login
Description	Allows a registered user to authenticate and access the system.
Precondition	The user has a registered account.
Postcondition	The user is authenticated and the main menu is displayed.
Main Flow	1. The system displays the login form. 2. The user enters username and password. 3. The system validates the credentials. 4. Credentials are valid. 5. The system displays the main menu (Check Schedule, Check Prices, Reservation, Exit).
Alternative Path	3a. The credentials are invalid. → The system displays an error. The failed attempt counter increments. Resume at step 1.
Exceptions	At any point, the user may cancel. → The system returns to the welcome screen.

Table 14 - UC02: Login

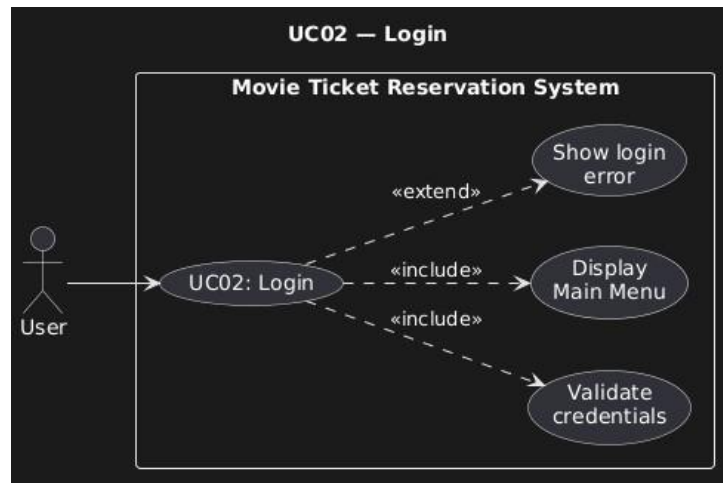


Figure 4 - UC2 Login

6.5 Use Case Specification UC03: Check Schedule

Actor	User (logged in)
Use Case Name	Check Schedule
Description	Allows the user to view the schedule of available movies with dates, times, and screening rooms.
Precondition	The user is logged in.
Postcondition	The movie schedule is displayed to the user.
Main Flow	1. The user selects Check Schedule from the main menu. 2. The system retrieves all future movie screenings. 3. The system displays a list of movies with date, time, and screening room. 4. The user views the schedule. 5. The user returns to the main menu.
Alternative Path	3a. No screenings are available. → The system displays a message indicating no movies are currently scheduled.

Table 15 - UC03: Check Schedule

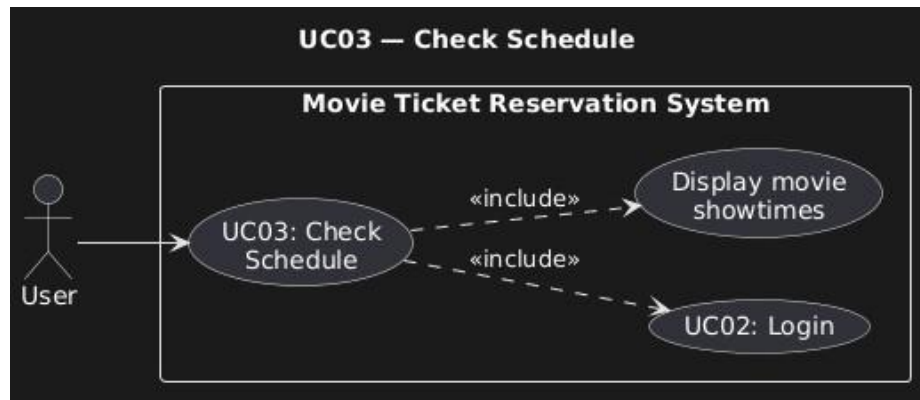


Figure 5 - UC3 Check Schedule

6.6 Use Case Specification – UC04: Check Prices

Actor	User (logged in)
Use Case Name	Check Prices
Description	Allows the user to consult ticket prices for different movies and seat types.
Precondition	The user is logged in.
Postcondition	Ticket prices are displayed to the user.
Main Flow	1. The user selects Check Prices from the main menu. 2. The system retrieves the price list for all movies. 3. The system displays prices per movie and per seat type (regular, VIP). 4. The system displays any active promotions or discounts. 5. The user views the prices. 6. The user returns to the main menu.
Alternative Path	3a. No promotions are currently active. → The system displays prices without a promotions section.
Exceptions	At any point, the user may return to the main menu.

Table 16 - UC04: Check Prices

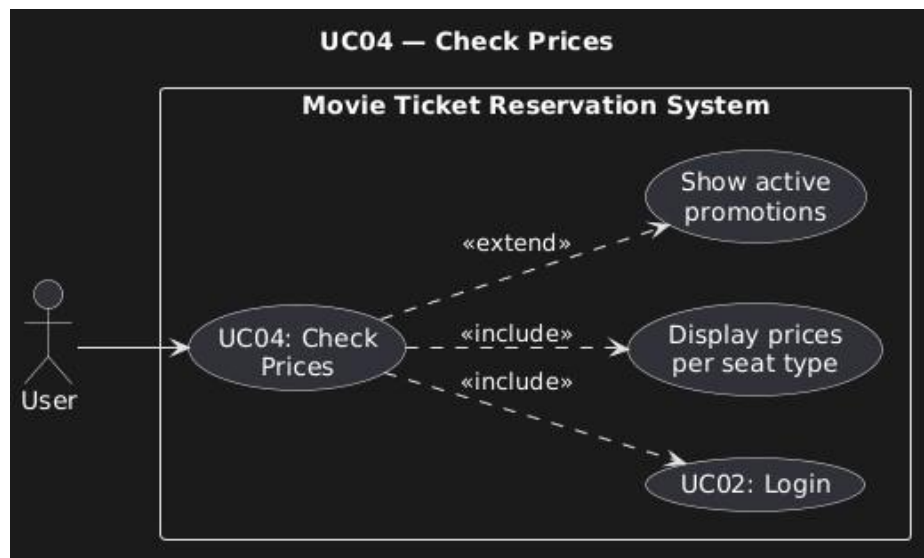


Figure 6 - UC4 Check Prices

6.7 Use Case Specification – UC05: Make a Reservation

Actor	User (logged in)
Use Case Name	Make a Reservation
Description	Allows the user to reserve seats for a movie through a guided flow: choose movie, specify tickets, select chairs, enter payment, and receive confirmation.
Precondition	The user is logged in. There are movies with available screenings and seats.

6.8 Use Case Specification – UC06: Exit

Actor	User (logged in)
Use Case Name	Exit
Description	Allows the user to safely log out and terminate the session.
Precondition	The user is logged in.
Postcondition	The user is logged out and the system returns to the welcome screen.
Main Flow	1. The user selects Exit from the main menu. 2. The system logs the user out. 3. The system discards any unsaved reservation in progress. 4. The system returns to the welcome/login screen.
Alternative Path	2a. A reservation is currently in progress. → The system asks the user to confirm they want to exit and discard the reservation. → If confirmed, the reservation is discarded and chairs are released. Resume at step 2. → If not confirmed, the system returns to the main menu.
Exceptions	

Table 18 - UC06: Exit

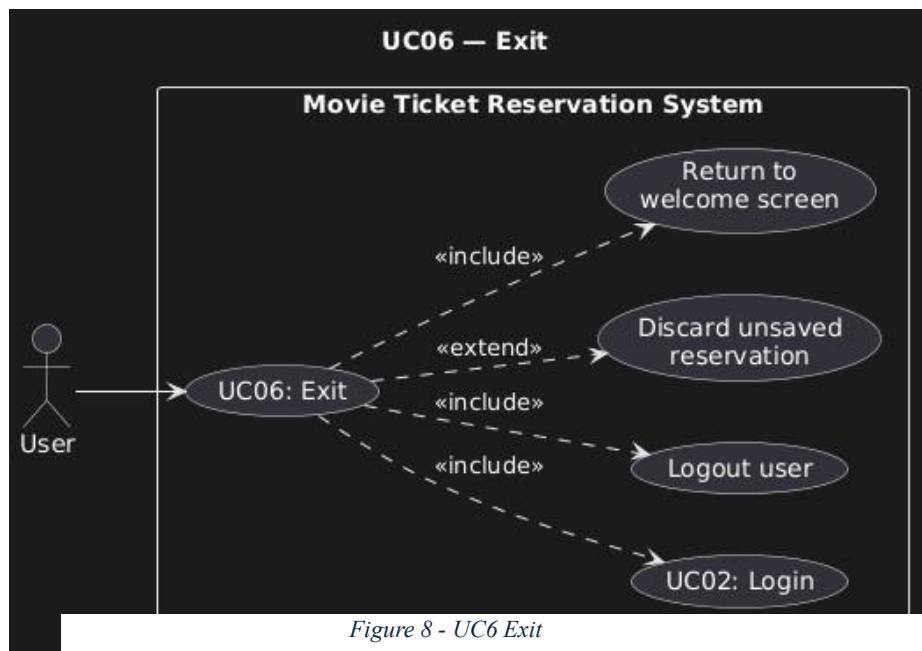


Figure 8 - UC6 Exit

7 Physical Architecture

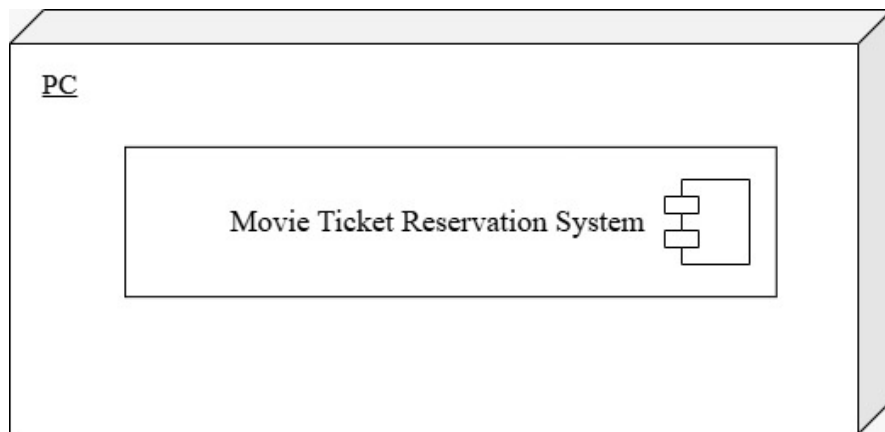


Figure 9 - Physical Architecture

8 Logical Architecture

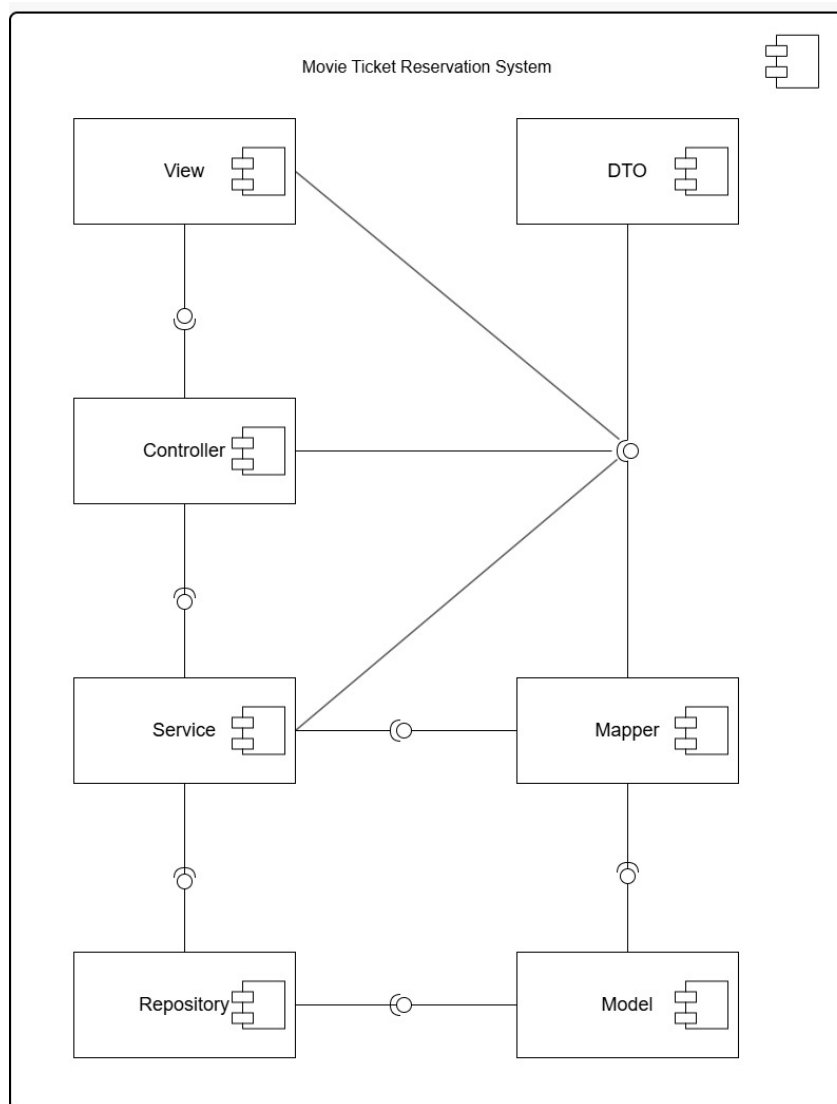


Figure 10 - Logical Architecture

9 Code Organization

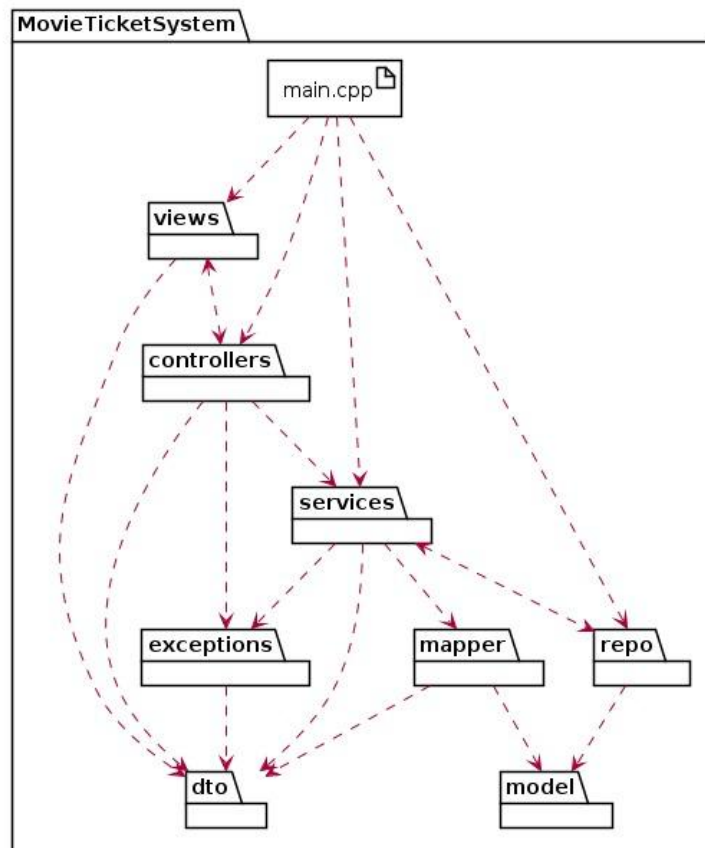


Figure 11 - Code organization

10 Model Class Diagram

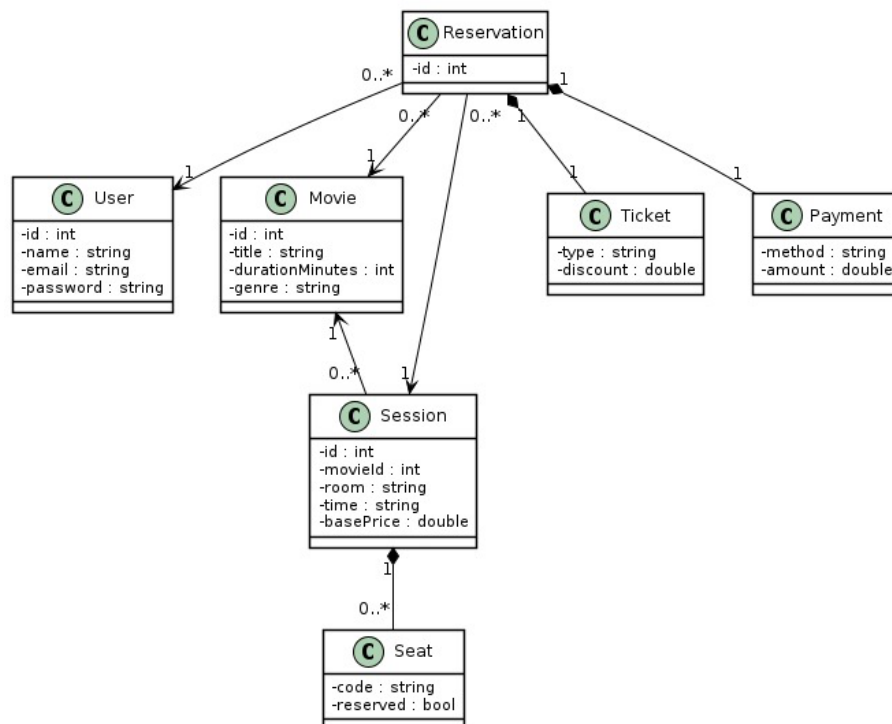


Figure 12 - Model Class Diagram

11 Exceptions Class Diagram

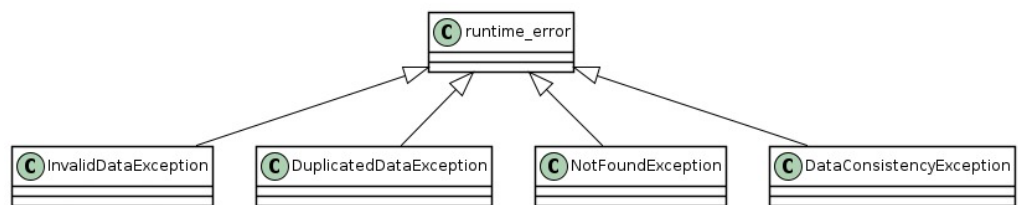


Figure 13 - Exceptions Class Diagram

12 Sequence Diagram

12.1 UC1 Register User

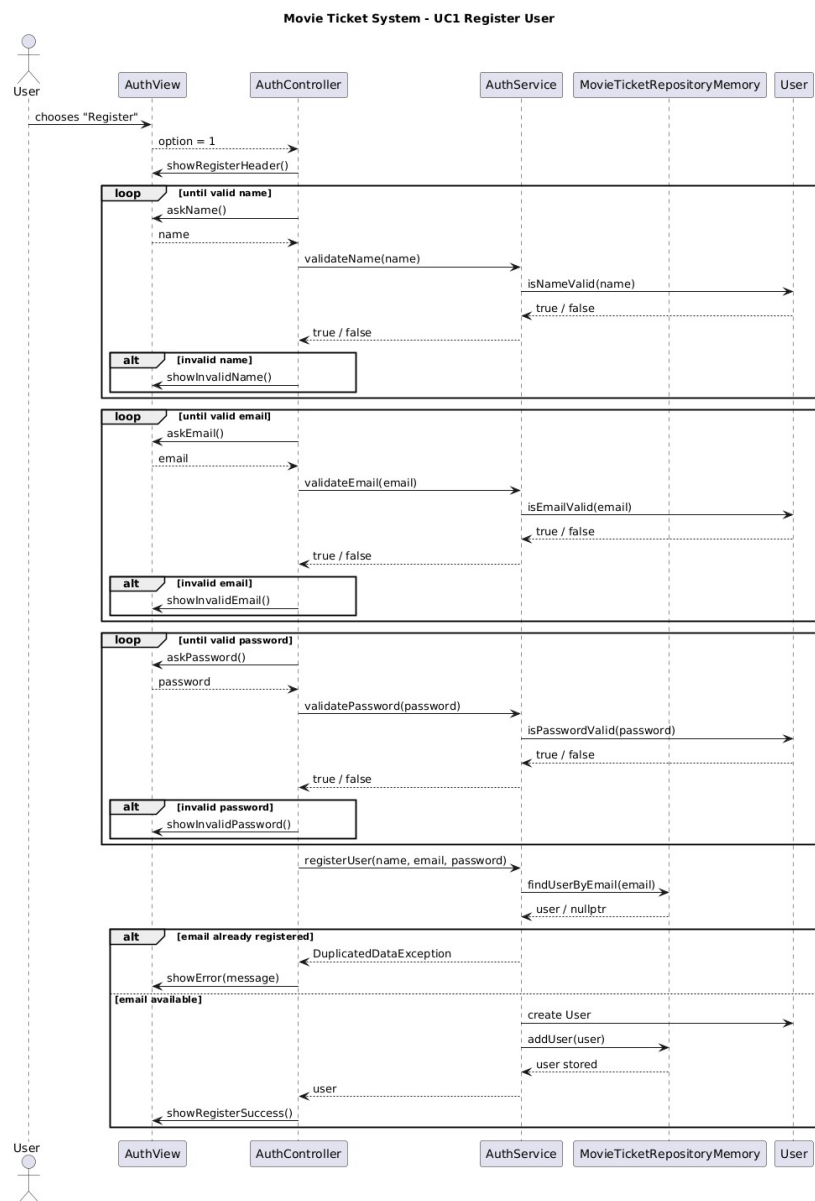


Figure 14 - UC1 Register User

12.2 UC2 Login User

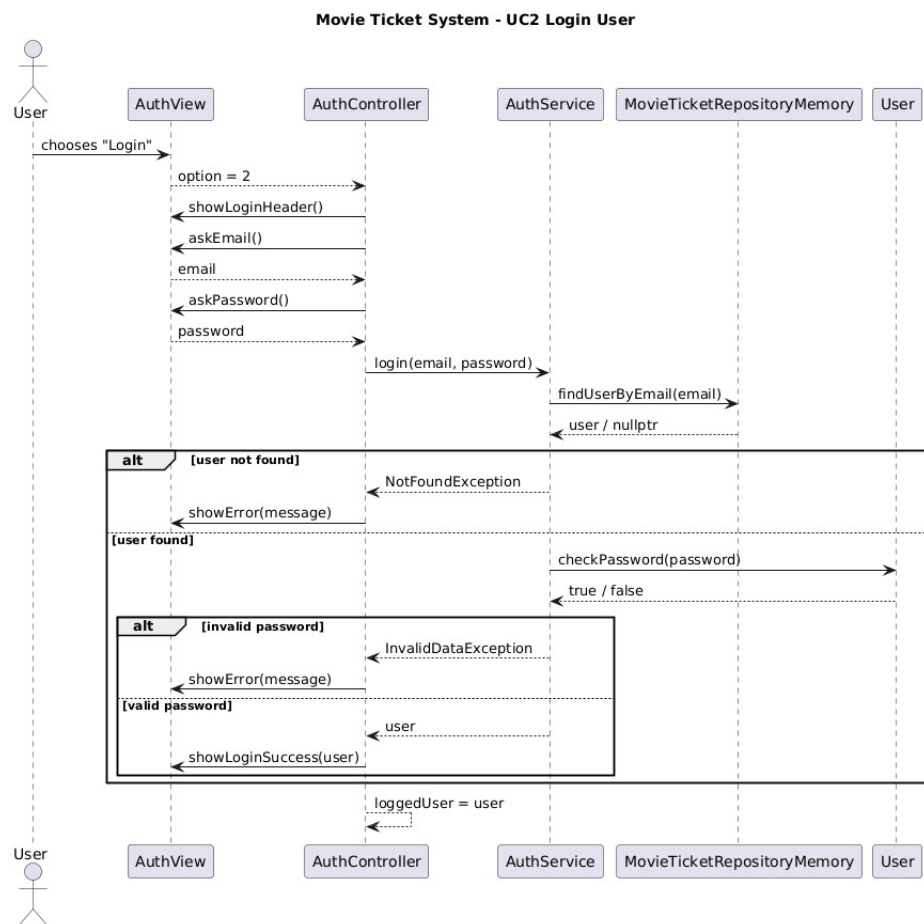


Figure 15 - UC2 Login User

12.3 UC3 Make Reservation

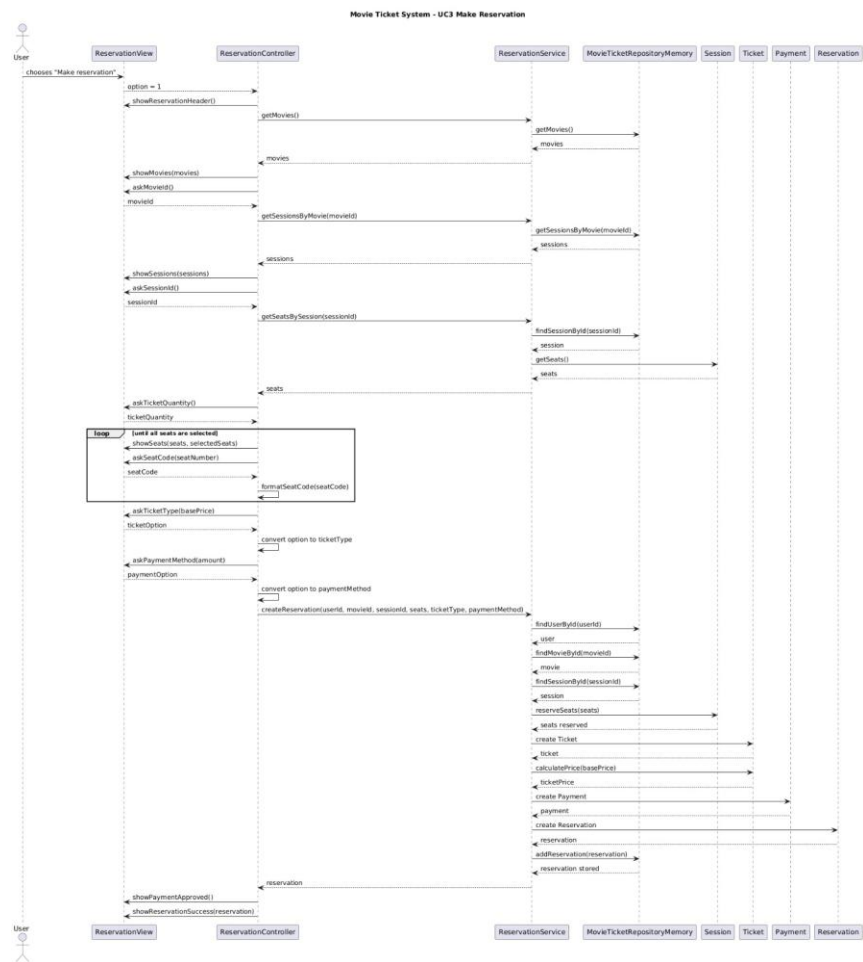


Figure 16 - UC3 Make Reservation

12.4 View My Reservations

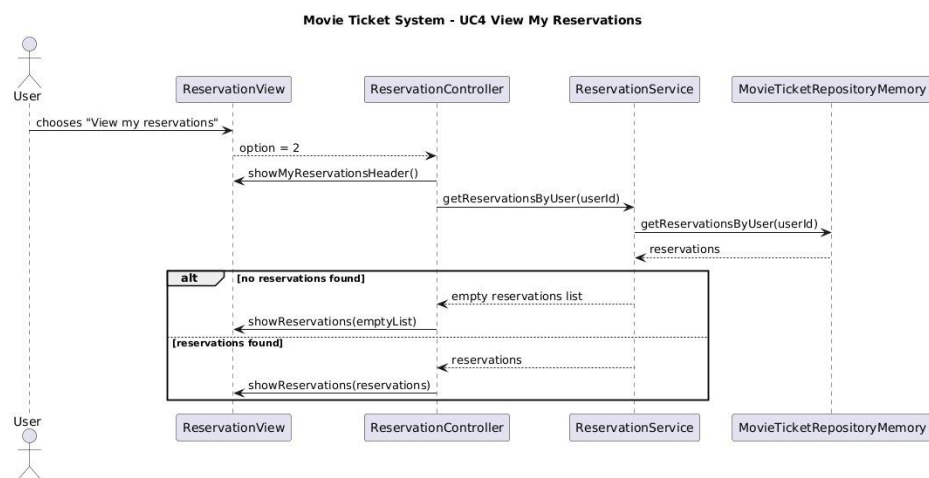


Figure 17 - View My Reservation

12.5 UC5 Logout

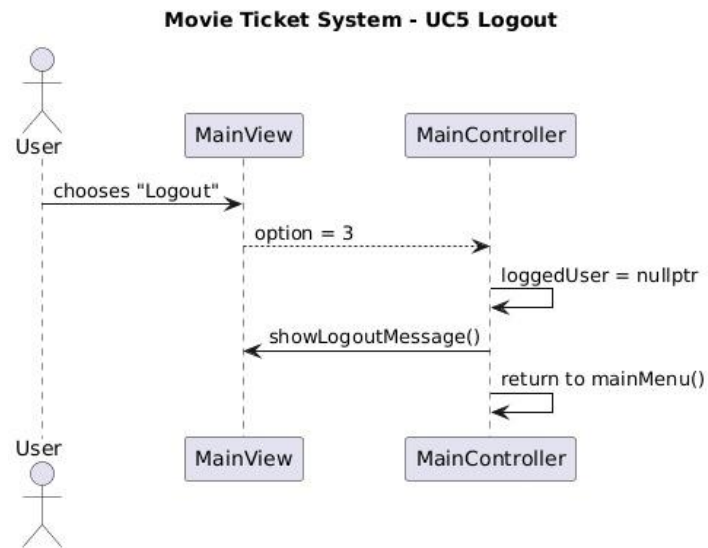


Figure 18 - UC5 Logout

13 User Interface

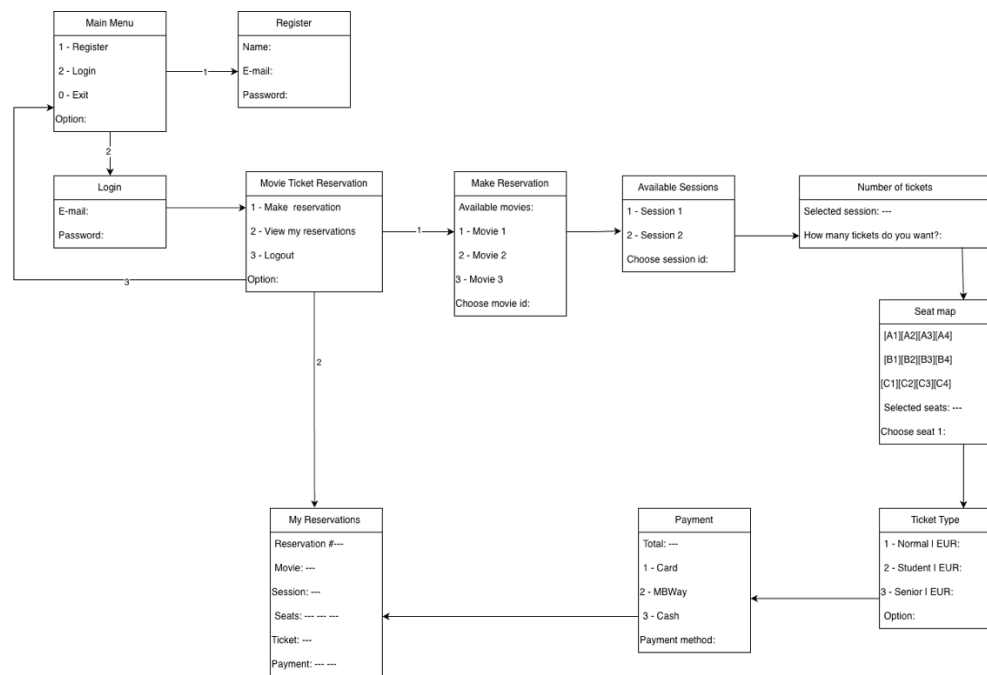


Figure 19 - User Interface

14 Implementation

14.1 Implementation Overview

The Movie Ticket Reservation System was implemented in C++ as a standalone terminal application. The implementation follows the designed layered architecture, separating the application into View, Controller, Service, Repository, DTO, Mapper, Model and Exceptions components.

This organization improves modularity and makes each class responsible for one main task. The View classes handle the terminal interface, the Controller classes coordinate the application flow, the Service classes contain the application logic, the Repository stores and retrieves data in memory, and the Model classes represent the domain objects of the system.

Layer / Package	Responsibility
View	Shows terminal menus and reads user input.
Controller	Coordinates the use case flow and calls the services.
Service	Contains application logic and validates business rules.
Repository	Stores and retrieves data in memory.
DTO	Transfers data between layers without exposing internal model objects.
Mapper	Converts model objects into DTO objects.
Model	Represents users, movies, sessions, seats, tickets, payments and reservations.
Exceptions	Handles invalid data, duplicated data, not found data and consistency problems.

Table 19 - Implementation Overview

14.2 Project Structure

The project is organized using separate header and implementation files. Header files are stored in the headers folder and source files are stored in the sources folder.

```
MovieTicketSystem/
├── main.cpp
├── CMakeLists.txt
├── headers/
│   ├── controllers/
│   ├── views/
│   ├── services/
│   ├── repository/
│   ├── dto/
│   ├── mapper/
│   ├── model/
│   └── exceptions/
├── sources/
│   ├── controllers/
│   ├── views/
│   ├── services/
│   └── repository/
```

mapper/
model/

14.3 Main Implementation Rules Applied

Rule	Application in the project
Header / source separation	Classes are declared in .h files and implemented in .cpp files.
Naming convention	Classes use CamelCase and methods/attributes use lowerCamelCase.
Single Responsibility Principle	Each layer has a clear responsibility: view, controller, service, repository, model.
Business rules in functions	Validation rules are implemented through methods such as isPasswordValid or reserveSeats.
Exceptions for invalid rules	InvalidDataException, DuplicatedDataException, NotFoundException and DataConsistencyException are thrown when needed.
DTO and Mapper patterns	DTOs carry data between layers and mappers convert model objects into DTOs.

Table 20 - Implementation Rules

14.4 Header and Implementation Separation

The following excerpt shows how a class is declared in a header file and implemented in a source file. This follows the rule that all classes are split between .h and .cpp files.

```
// User.h
class User {
private:
    int id;
    std::string name;
    std::string email;
    std::string password;

public:
    User(int id, const std::string& name,
         const std::string& email,
         const std::string& password);

    void setName(const std::string& name);
    static bool isValidName(const std::string& name);
};
```

```
// User.cpp
bool User::isValidName(const std::string& name) {
    return name.length() >= 3;
}

void User::setName(const std::string& name) {
    if (!isValidName(name)) {
        throw InvalidDataException("Invalid user name");
    }

    this->name = name;
}
```

The attributes are private because the internal state of the object must be protected. Other classes cannot directly change the name, email or password without using methods that validate the data first. The public methods provide controlled access to the object behavior.

14.6 Code Example: Business Rule and Exception

Business rules are implemented through functions. When a rule is not fulfilled, an exception is thrown. The password rule is one example of this approach.

```
bool User::isPasswordValid(const std::string& password) {
    if (password.length() < 8) {
        return false;
    }

    bool hasUppercase = false;
    bool hasLowercase = false;
    bool hasNumber = false;

    for (char character : password) {
        if (std::isupper(character)) hasUppercase = true;
        if (std::islower(character)) hasLowercase = true;
        if (std::isdigit(character)) hasNumber = true;
    }

    return hasUppercase && hasLowercase && hasNumber;
}
```

Table 21 - Code Example Business Rule and Exception

14.7 Code Example: Repository

The repository is responsible for data storage and retrieval. In this project, the concrete repository stores data in memory using vectors.

```
class MovieTicketRepositoryMemory : public IMovieTicketRepository {
private:
    std::vector<User*> users;
    std::vector<Movie> movies;
    std::vector<Session> sessions;
    std::vector<Reservation*> reservations;

public:
    std::vector<User*>& getUsers() override;
    std::vector<Movie>& getMovies() override;
    std::vector<Session>& getSessions() override;
    std::vector<Reservation*>& getReservations() override;
};
```

Table 22 - Code Example Repository

The service layer accesses data through the repository interface instead of depending directly on a specific storage implementation. This reduces coupling and makes it possible to replace the memory repository with a file or database repository in the future.

14.8 Code Example: Service Layer

The service layer contains application logic. The controller asks the service to perform operations, and the service uses the repository and model classes to validate and execute the requested behavior.

```
UserDTO AuthService::login(const std::string& email,
                           const std::string& password) {
    User* user = searchByEmail(email);

    if (user == nullptr) {
        throw NotFoundException("User not found");
    }

    if (!user->checkPassword(password)) {
        throw InvalidDataException("Invalid password");
    }

    return UserMapper::modelToDTO(user);
}
```

Table 23 - Code Example Service Layer

This example shows why the login logic is placed in the service instead of the controller. The controller coordinates the flow, while the service validates the credentials, handles exceptions and returns the result.

14.9 Code Example: DTO and Mapper

DTOs are used to transfer data between layers. Mappers convert model objects into DTOs. This prevents controllers and views from depending directly on internal model objects.

```
UserDTO UserMapper::modelToDTO(User* user) {
    UserDTO dto;

    dto.id = user->getId();
    dto.name = user->getName();
    dto.email = user->getEmail();

    return dto;
}
```

Table 24 - Code Example DTO and Mapper

The password is not copied to the UserDTO. This means the presentation layer can show user information without accessing sensitive internal data.

15 Testing

15.1 Testing Approach

Testing was planned according to the Arrange-Act-Assert structure presented in TP9. In this structure, each test first arranges the necessary objects and inputs, then acts on the behavior being tested, and finally asserts the expected result.

The testing strategy includes unit-oriented tests for model and service behavior, and integration testing by executing the main use cases through the application.

15.2 Code Example: Arrange-Act-Assert Test

The following example shows how a unit test can be structured using Arrange-Act-Assert.

```
TEST(UserTest, ValidPassword) {  
    // Arrange  
    std::string password = "Password1";  
  
    // Act  
    bool result = User::isPasswordValid(password);  
  
    // Assert  
    EXPECT_TRUE(result);  
}
```

Table 25 - Code Example Arrange-Act-Assert Test

The Arrange step prepares the input, the Act step executes the behavior under test, and the Assert step verifies that the result is the expected one.

15.3 Unit-Oriented Test Cases

Test ID	Class / Function	Purpose	Expected Result
UT01	User::isNameValid	Validate a name with at least 3 characters	Returns true
UT02	User::isNameValid	Validate a name with less than 3 characters	Returns false
UT03	User::isEmailValid	Validate an email containing @ and .	Returns true
UT04	User::isPasswordValid	Validate a strong password	Returns true
UT05	User::isPasswordValid	Validate a weak password	Returns false
UT06	Session::isSeatAvailable	Check if a free seat can be selected	Returns true
UT07	Session::reserveSeats	Reserve a valid seat	Seat becomes unavailable
UT08	Ticket::calculatePrice	Apply ticket discount to base price	Returns correct price

UT09	AuthService::login	Login with correct credentials	Returns user data
UT10	AuthService::login	Login with invalid credentials	Throws exception

Table 26 - Unit-Oriented Test Cases

15.4 Integration Testing by Use Case

Use Case	Percentage	Notes
UC1 - Register User	100%	Registration works with valid data and rejects invalid/duplicated data.
UC2 - Login User	90%	Login works, but account lock after three failed attempts is not implemented.
UC3 - Make Reservation	90%	Reservation flow works, but payment card validation is simplified.
UC4 - View My Reservations	100%	The system displays reservations created by the logged-in user.
UC5 - Logout	100%	The system logs out and returns to the main menu.

Table 27 - Integration Testing by Use Case

15.5 Functional Test Cases

Test ID	Feature	Steps	Expected Result	Result
T01	Register user	Enter valid name, email and password	Account is created successfully	Pass
T02	Invalid name	Enter a name with less than 3 characters	Error message is displayed	Pass
T03	Invalid email	Enter email without @ or .	Error message is displayed	Pass
T04	Invalid password	Enter weak password	Error message is displayed	Pass
T05	Duplicate email	Register two users with the same email	System prevents duplicate registration	Pass
T06	Login valid user	Enter correct email and password	Authenticated menu is displayed	Pass
T07	Login invalid email	Enter an email not registered	Error message is displayed	Pass
T08	Login invalid password	Enter wrong password	Error message is displayed	Pass
T09	Make reservation	Select movie, session, tickets, seats, ticket type and payment	Reservation is created	Pass
T10	Unavailable seat	Try to select an already reserved seat	System rejects the seat	Pass
T11	Invalid ticket quantity	Enter 0 or more than 8 tickets	Error message is displayed	Pass
T12	View reservations	Create a reservation and choose View my reservations	Reservation details are displayed	Pass

T13	Logout	Select logout option	User returns to main menu	Pass
-----	--------	----------------------	---------------------------	------

Table 28 - Functional Test Cases

15.6 Testing Conclusion

The tests confirm that the main flow of the system works correctly: user registration, login, movie and session selection, seat selection, ticket type selection, payment method selection, reservation creation, reservation visualization and logout. The system also handles invalid input through exceptions and error messages. Some requirements were simplified due to project scope, such as persistent storage, password encryption, account locking and real payment validation.

16 Glossary

Term (EN)	Termo (PT)	Description
Account	Conta	A user record in the system containing username, email, and password.
Booking Reference	Referência de Reserva	A unique alphanumeric code identifying a reservation.
Chair	Cadeira/Lugar	An individual seating position in a screening room, identified by row and number.
CVV	CVV	Card Verification Value. A 3-digit security code on the payment card.
Exit	Sair	The action of logging out and terminating the user session.
Genre	Gênero	A category classifying a movie (e.g., Action, Comedy, Drama).
Login	Iniciar Sessão	The process of authenticating a user with username and password.
Main Menu	Menu Principal	The screen displayed after login with options: Schedule, Prices, Reservation, Exit.
Movie	Filme	A film available for screening at the theater.
Payment	Pagamento	A financial transaction using card details to confirm a reservation.
Price	Preço	The monetary cost of a ticket, which may vary by seat type.
Rating	Classificação	An age-based classification assigned to a movie (e.g., PG, PG-13, R).
Registration	Registo	The process of creating a new user account in the system.
Reservation	Reserva	A confirmed booking of specific chairs for a movie screening.
Reserved Screen	Ecrã de Confirmação	The confirmation screen shown after a successful reservation.
Schedule	Horário	The list of movie screenings with dates, times, and rooms.
Screening Room	Sala de Cinema	A physical room in the theater where movies are projected.
Seating Map	Mapa de Lugares	A visual layout of chairs in a screening room showing availability.
SSD	SSD	System Sequence Diagram. A UML diagram showing actor-system interaction.
Ticket	Bilhete	A purchased item granting access to a specific chair at a screening.
UC	UC	Use Case. A description of a system function triggered by an actor.
User	Utilizador	A registered person who can access system features after login.

VIP	VIP	A premium seat type with additional comfort, typically at a higher price.
-----	-----	---

Table 29 - Glossário

References

1. Larman, C.: Applying UML and Patterns. Prentice Hall (2004)
2. Sommerville, I.: Software Engineering. 10th edn. Pearson (2015)
3. Cockburn, A.: Writing Effective Use Cases. Addison-Wesley (2000)
4. Cohn, M.: User Stories Applied. Addison-Wesley (2004)
5. Design TP6 Fundamentos de Desenvolvimento de Software (FSOFT) PBS (2025/2026)
6. Implementation TP8 Fundamentos de Desenvolvimento de Software (FSOFT) PBS (2025/2026)
7. Testing TP9 Fundamentos de Desenvolvimento de Software (FSOFT) PBS (2025/2026)